AI DATA FOUNDRY

Product, Architecture, Vercel UX & Codex Implementation Master Plan

Version: 2.0 — Codex implementation edition (supersedes V1 planning baseline)
Date: 4 November 2026
Status: Full engineering specification and Codex handoff; implementation not yet started
Intended readers: Product leadership, AI/ML researchers, platform engineers, infrastructure operators, security reviewers, and implementation agents
Design direction: Original Vercel Geist-inspired interface; production API parity; no nonfunctional controls.
North star: A unified, API-first operating system for creating, curating, simulating, governing, and exporting multimodal AI training data.

> ***Executive position.*** *Build the data lifecycle control plane, not another monolithic ML framework. A researcher describes the dataset they need; the platform plans and executes permitted generation and ingestion jobs across models, simulators, and compute pools; validates the results; publishes an immutable version; and measures whether the dataset improves downstream model performance. The same capabilities must work from the web studio, REST API, SDK, and CLI.*

Planning assumptions. Initially target a small engineering team and a tightly scoped first production release. Third-party API accounts, cloud infrastructure, hardware clusters, legal approvals, and commercial licenses are *not* assumed available. Specific implementation versions and compatibility must be pinned and tested in the repository rather than inferred from this document. All costs, dates, and reliability targets below are goals or estimates—not statements of actual delivery.

---

1. Executive summary

1.1 Problem

AI teams build training datasets through fragmented systems: Python scripts for ingestion, API wrappers for model-generated examples, separate labeling products, distributed data processors, 3D simulation stacks, object stores, GPU schedulers, experiment trackers, and spreadsheets recording provenance. Fragmentation creates inconsistent schemas, expensive reruns, data leakage, incomplete licensing records, and an inability to reproduce exactly what trained a model.

1.2 Product promise

AI Data Foundry offers **one contract for creating training data**, regardless of modality or execution environment. Users specify *what* they want; the platform records *how* it was produced, *where* it was processed, *why* it passed quality gates, and *which version* was supplied to training.

Core use cases:

1. Generate structured LLM instruction, preference, tool-use, classification, and independently verifiable problem/solution data using authorized teacher models.
2. Produce synthetic images, videos, audio, text, and multimodal examples through generation providers or self-hosted models.
3. Create robotics and perception datasets from Blender, NVIDIA Isaac Sim, Omniverse Replicator, and Isaac Lab.
4. Ingest and curate user-provided datasets at TB scale with labeling, deduplication, filtering, classification, and human review.

5. Schedule jobs across local CPUs, RTX rendering nodes, general-purpose GPUs, accelerators, Kubernetes clusters, Slurm-managed supercomputers, and approved cloud providers.
6. Evaluate dataset distributions, contamination, source rights, labeling quality, and model impact; use failure analysis to propose targeted generation.
7. Provide complete API/SDK/CLI automation, webhooks, governed exports, and integration with customer training systems.

1.3 Strategic product boundary

Own: Dataset recipe language, canonical metadata, versioning and provenance, workspace permissions, approvals and budgets, orchestration semantics, compute capability matching, quality gates, studio workflows, API contracts, and consumption/billing metering.

Integrate: Foundation model inference, image/audio generation, rendering, physics engines, robotic training environments, distributed dataframe engines, container scheduling, cluster managers, and downstream model training frameworks.

Do not build initially: An in-house foundation model, GPU hypervisor, replacement for Kubernetes/Slurm, a custom 3D rendering engine, full-scale distributed training service, universal marketplace, or autonomous unlimited-cost AI agent.

1.4 Launch definition

Production V1 is a secure, metered, API-first platform where an authenticated organization can upload or register source material; define a deterministic dataset recipe; start and monitor a job; enforce source-use policy and budgets; run ingestion/annotation/generation/quality steps; publish an immutable version with provenance; and export it through signed URLs or integrations. The initial path can run on CPU and provider APIs. Simulation and advanced HPC are staged integrations, not prerequisites to a working vertical slice.

2. Users, jobs to be done, and differentiation

Persona	Primary job	Success signal
LLM research engineer	Prepare verified SFT/preference/evaluation candidates from permitted sources	Higher held-out quality per dollar
Vision researcher	Build balanced image/video data with accurate labels	Class coverage and annotation accuracy
Robotics engineer	Generate episodes with synchronized sensors/actions/rewards	Successful sim-to-real transfer and reproducibility
ML platform engineer	Schedule, audit, and reproduce multi-step data pipelines	Fewer failed jobs and less manual intervention
Data operations/annotation lead	Review uncertain labels and enforce data policies	Faster review, fewer accepted errors
Security/compliance owner	Control licenses, PII, retention, access, and audit	Complete evidence and enforceable policy
SaaS developer	Embed data generation and dataset export into an application	Stable documented API and predictable cost

Differentiation should rest on: (a) multimodal generation plus curation in the same dataset lifecycle, (b) simulation integrated as a first-class data source, (c) compute-agnostic control plane with *capability-aware* worker selection, (d) provenance as an invariant, and (e) closed-loop evaluation that measures model improvement rather than sample volume.

3. Product requirements and priorities

P0 — First production release: Multi-tenant organization/workspace model; service accounts and scoped API keys; datasets, versions, artifacts, manifests, and recipe validation; upload/import; asynchronous workflows with retries; at least one licensed generation adapter; CPU processing; basic annotation and review; quality gates; signed exports; OpenAPI + Python SDK; job budgets; audit logs; monitoring; CI/CD; backup/restore and documented incident procedures.

P1 — Product expansion: Additional teacher and multimodal generators; domain-aware labeling; dataset search/embeddings; distributed Ray processing; RTX Blender and Isaac Sim workers; Isaac Lab episode workflows; Slurm/Kubernetes compute agents; richer visual analytics; TypeScript SDK and CLI; multi-region storage policy; evaluation feedback integrations.

P2 — Advanced differentiators: Dataset Architect AI, active-learning loop, autonomous (approval-gated) recipe suggestions, plugin SDK and marketplace, on-prem control plane, hybrid tenancy, advanced data attribution, licensed dataset exchange, custom accelerators, public ecosystem.

3.1 Functional specification: twelve modules

Module	P0 scope	P1 / P2 depth
Generation Engine	Provider abstraction, structured text generation, budgets, sample provenance	Image/audio/video, self-hosted model serving, multimodal alignment
Teacher Studio	Authorized teacher connector, JSON schema output, validation	Multi-teacher voting, preference data, reasoning-task verification
Simulation Studio	Recipe schemas + adapter protocol + sample local Blender job	Remote Blender, Isaac Sim/Replicator, Isaac Lab, sensor synchronization
Labeling Studio	Task queues, machine suggestions, reviewer corrections, export	Consensus, active learning, specialized 3D/video/robotic annotation
Data Intelligence	Rule validators, exact/near dedupe, schema checks, quality report	Semantic dedupe, PII models, coverage and drift analysis
Dataset Registry	Immutable versions, data cards, lineage manifests, signed export	Global catalog, semantic search, cross-project access
Compute Fabric	Local worker, quota/budget guard, capability fields	K8s/Kueue, Ray, Slurm, private agents, multi-cloud routing
Workflow Engine	Recipe DAG, durable run state, retries, checkpoints	Branches, scheduled runs, templates, reusable components
Evaluation Lab	Dataset quality metrics and immutable evaluation results	Training-feedback import, benchmark analysis, targeted

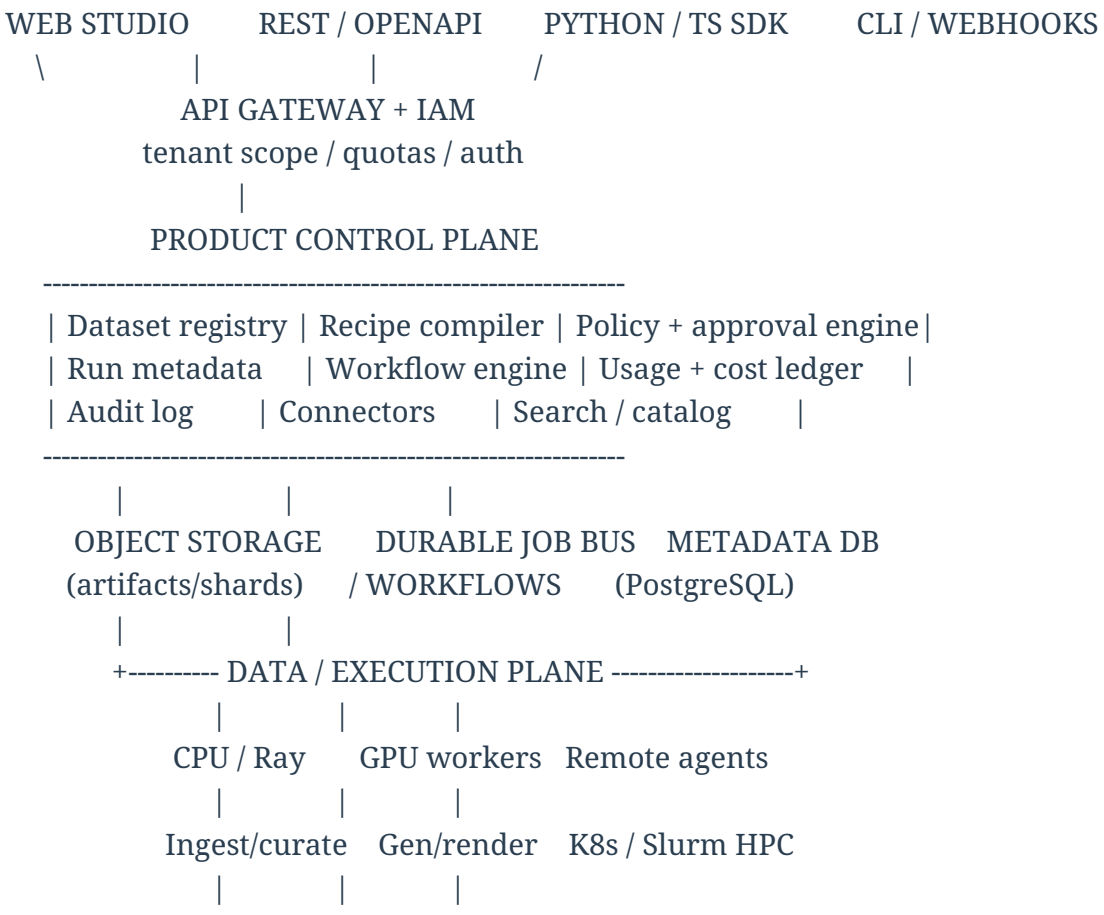
Module	P0 scope	P1 / P2 depth
		data generation
Developer Platform	REST/OpenAPI, Python SDK, signed webhooks, idempotency	TypeScript SDK, CLI, MCP interface, Terraform provider
Security/Governance	Tenant isolation, RBAC, secrets, source-use policies, audit	SSO/SCIM, region residency, customer-managed keys, private networking
Operations/Billing	Metering, cost estimation, spend caps, alerts, support tooling	Chargeback, invoicing, enterprise contracts, reservation pricing

3.2 Explicit non-goals for V1

- Unreviewed autonomous training-data generation using unapproved commercial model outputs.
- Full graphical 3D editing of arbitrary high-complexity scenes in a browser; begin with remote preview and parameterized scene recipes.
- Guarantees that synthetic examples are true, unbiased, or sufficient to improve model performance.
- Running every simulator on any GPU. Capabilities must be validated per engine and node.
- Storing all data rows or binary assets in PostgreSQL.
- Pushing arbitrary code directly from user recipes onto host machines without sandboxing and policy.

4. System architecture and trust boundaries

4.1 Logical architecture



validated outputs + measurements
|
immutable version + provenance
|
export / SDK consumption / training systems

4.2 Control plane vs. data plane

The **control plane** owns identity, organization boundaries, dataset records, policies, scheduling decisions, recipe compilation, billing, and metadata. It must not assume direct access to every customer's raw data. The **data plane** performs execution within isolated workers, transfers approved artifacts, performs quality checks, and reports events/results.

A private compute agent should make **outbound-only, mutually authenticated** connections to the control plane. It should obtain short-lived job credentials and scoped work instructions, not a global API key or blanket cloud credentials. Customer-owned environments should support local data access and optional customer-owned storage to avoid unnecessary egress.

4.3 Initial technology decisions

Concern	Default	Rationale / caveat
Frontend	Next.js, TypeScript, React, React Flow	Data studio, job progress, recipe DAG
API service	FastAPI + Pydantic + OpenAPI	Typed contracts, Python ecosystem
Database	PostgreSQL with tenant-aware access control	Transactional metadata; enforce app and DB boundaries
Object storage	S3-compatible, multipart and signed URLs	TB-scale assets; immutable version manifests
Durable workflows	Temporal (self-hosted or managed)	State/retries/long-running approval steps
Small-job transport	Workflow task queues; events via transactional outbox	Avoid two competing orchestrators initially
Distributed data	Ray Data when P1 scale requires it	Batch inference and curation at CPU/GPU scale
GPU workloads	Kubernetes Jobs + Kueue	Quotas, admission control, heterogeneous resources
HPC	Slurm adapter with job poller and checkpoints	Existing supercomputing environments
Simulation	Blender; Isaac Sim/Replicator; Isaac Lab	Keep versions/runtimes isolated
Curation	NeMo Curator + native validation operators	Reuse existing modality tools
Lineage	Internal manifest + OpenLineage events	Cross-system provenance
Observability	OpenTelemetry, Prometheus, Grafana, error tracking	Traces, logs, metrics, alerting

Concern	Default	Rationale / caveat
IaC/CI	Terraform, GitHub Actions, Docker	Reproducible deployment and tests

Architectural recommendation: Start as a *modular monolith API plus separate workflow and worker processes*. Split services only when independent deployment, security boundaries, throughput, or team ownership justifies it. Avoid operating Kafka, Flink, ClickHouse, Trino, and a search cluster in V1 without specific demand.

5. Canonical objects, lifecycle, and data model

5.1 Domain objects

- **Organization:** Billing, identity, data-residency and policy boundary.
- **Workspace/Project:** Permissions and isolation for a research initiative.
- **Dataset:** Stable logical identity, modality, ownership, schema and lifecycle state.
- **DatasetVersion:** Immutable published snapshot containing manifest hash, source references, sample counts, quality results, legal flags, and export locations.
- **Sample:** Logical entry with modality-specific typed fields and artifact references; stored in Parquet/JSONL or structured shards rather than a large relational row table.
- **Artifact:** Content-addressed object-store blob with size, media type, encryption, checksum and retention class.
- **Recipe:** Validated declarative DAG specifying generators, transformations, gates, and exports.
- **RecipeRevision:** Immutable recipe definition and engine compatibility lock.
- **Run:** Execution of a recipe revision with state, budgets, progress, worker allocations and lineage.
- **RunStep/Attempt:** Retriable unit of work with idempotency and checkpoint state.
- **ProviderConnection:** Secret reference, approved capabilities, terms/license tags and quotas.
- **ComputePool/Node:** Owner, scheduler type, available capabilities, data-residency attributes, pricing, health.
- **AnnotationTask/Review:** Label proposal, reviewer decision, disagreement, adjudication, audit.
- **Evaluation:** Versioned metric computation, benchmark identifiers, threshold results.
- **Export:** Format/target, signed download or destination transfer, permitted-use checks.
- **Policy/Approval/AuditEvent/UsageEvent:** Governance, workflow controls and billing evidence.

5.2 Database relationship sketch

```
organizations 1--n workspaces 1--n datasets 1--n dataset_versions
      |              |                |          |
      |              |                |      manifests -> artifacts
      |              +--n recipes 1--n recipe_revisions
      |                               |
      +--n provider_connections           +--n runs 1--n run_steps
      |                                |
      +--n compute_pools 1--n nodes           usage_events
      |                                |
      +--n policies + audit_events          evaluations + approvals
```

Prefer PostgreSQL UUIDs or ULIDs for internal identifiers; public IDs should be opaque and non-sequential. Every tenant-owned row requires `organization_id`; authorization must never trust IDs supplied by a client without ownership checks. Financial usage events should be append-only and reconcile to execution records.

5.3 State machines

Run: CREATED -> VALIDATING -> AWAITING_APPROVAL -> QUEUED -> RUNNING -> VERIFYING -> PUBLISHING -> SUCCEEDED. Terminal alternatives: FAILED, CANCELLED, EXPIRED, BLOCKED_POLICY. PAUSED and RESUMING apply only when the engine supports checkpoints. No transition may bypass budget/policy gates.

Dataset version: DRAFT -> VALIDATING -> READY -> PUBLISHED -> ARCHIVED; blocked validation stays in REJECTED, with immutable evidence. Once published, bytes and manifest contents must not mutate—corrections create another version.

Annotation: NEW -> AUTO_LABELED -> NEEDS_REVIEW -> ACCEPTED/REJECTED -> ADJUDICATED, with separate handling for directly accepted high-confidence samples when policy permits.

6. Dataset format and provenance specification

6.1 Storage conventions

Use object storage for raw and processed binary data; PostgreSQL for metadata and indexes. Proposed path convention:

```
org/<org-id>/projects/<project-id>/datasets/<dataset-id>/
  sources/<source-id>/...
  staging/runs/<run-id>/...
  versions/<version-id>/manifest.json
  versions/<version-id>/data/part-00001.parquet
  versions/<version-id>/media/...
  versions/<version-id>/quality/report.json
  versions/<version-id>/provenance/lineage.jsonl
  exports/<export-id>/...
```

Use multi-part upload, sha256 checksums, object locking where required, content-addressed assets, resumable download, and lifecycle rules. Object keys are not authorization boundaries; check policy before issuing signed links. Store source material and generated output in distinct buckets/prefixes to simplify rights and retention enforcement.

6.2 Universal sample envelope

Each sample should have these consistent fields even when the payload is modality-specific:

Field	Description
sample_id, dataset_version_id	Stable identity and source version
modality, schema_version	Type and validation contract
payload_ref, annotation_refs	Object reference(s) and typed label objects
source_refs, generation_run_id	Origin and producing process
model_engine, model_engine_version	Exact generating engine/model revision
prompt_template_ref, seed	Reproduction inputs where retention is permitted

Field	Description
license_policy_id, consent_flags	Use constraints and rights state
quality_metrics, review_status	Acceptance evidence
content_hash, created_at	Integrity and timestamp

For text, export JSONL/Parquet with prompt, response, task tags, verifier results, and data policy. For vision, support image/media pointers and COCO/YOLO labels. For robotics, support time-aligned observations, actions, rewards, terminations, episode metadata, and sensor calibration; provide a **LeRobot v3 compatible exporter** once tested. Preserve timestamps and units in all multistream datasets. For general discoverability expose **MLCommons Croissant 1.1** metadata as an export option.

6.3 Provenance manifest

The manifest is immutable and signed or hash-sealed at publication. It includes dataset ID/version, content checksums, artifact list, recipe revision, source licenses and policies, teacher-model permission record, generator/provider details, model/engine version, worker image digest, code revision, deterministic seeds (when feasible), annotation history, quality/evaluation results, PII handling, and the publication approval signature.

Reproducibility has levels: Metadata-reproducible (same inputs, versions, parameters); deterministically reproducible (identical outputs for supported engines); and statistically reproducible (nondeterministic generators reproduce distributions/tolerances). Do not promise bit-for-bit reproduction of external model APIs or nondeterministic GPU simulators.

7. Dataset recipes and workflow DSL

A **recipe** is a declarative, versioned DAG. Each node names a registered engine type and a schema-validated configuration. References to secrets use connection IDs, never raw credentials embedded in YAML. Every run should freeze recipe revision, approved provider connection configuration, selected model version, and dataset targets.

```yaml
apiVersion: foundry.ai/v1alpha1
kind: DatasetRecipe
metadata:
  name: warehouse-perception
  project: robotics-lab
spec:
  dataset:
    modality: robotics
    target_episodes: 50000
  budget:
    max_usd: 800
    approval_above_usd: 200
  compute:
    required_capabilities: [nvidia-rtx, isaac-sim-compatible]
    region_allowlist: [approved-region-1]
  steps:
    - id: make_scenes
      uses: blender.procedural@1
      with: {scene_template: warehouse_v2, seeds: 250}
    - id: simulate
```

```
    needs: [make_scenes]
    uses: isaacsim.replicator@1
    with: {episodes_per_scene: 200, sensors: [rgb, depth, lidar]}
  - id: validate
    needs: [simulate]
    uses: quality.robotics@1
    with: {require_time_alignment: true, max_invalid_ratio: 0.01}
  - id: publish
    needs: [validate]
    uses: dataset.publish@1
    with: {export_format: lerobot_v3}
```

This is a proposed Foundry recipe interface, not a native configuration accepted by Blender or Isaac.

Recipe compiler responsibilities: validate syntax; detect DAG cycles; resolve connector permissions; check model terms and modality compatibility; estimate cost; resolve resources; require approval; create execution graph; stage artifacts; enforce per-step timeouts and retry policy; publish only after quality gates pass. Allow human intervention steps and safe cancellation without silently discarding partial outputs.

8. API-first developer platform

8.1 Design rules

- Versioned REST API at `/v1`; JSON request/response; OpenAPI 3.1 as the contract of record.
- OAuth/OIDC for users and organizations; scoped API keys/service principals for automation; expiring signed upload/download URLs.
- Consistent response errors with `code`, `message`, `request_id`, and `details`; never expose provider secrets or cross-tenant metadata.
- All long operations return `202 Accepted` and an `operation_id` or `run_id`; never hold an HTTP connection for multi-hour processing.
- **Idempotency-Key** is required for chargeable `POST` operations. Same key + same request returns the original outcome; same key + changed payload returns a conflict.
- Support pagination (`limit`, `cursor`), sorting, filtering, request tracing, configurable rate limits, fair-use quotas and budget caps.
- API versions must be backwards compatible within major version; announce deprecations and expose schema compatibility tests.
- Emit signed CloudEvents-style webhooks with timestamp, delivery ID, replay protection, exponential backoff, dead-letter capture and redelivery controls.

8.2 Main endpoints

Area	Method and route	Operation
Identity	`GET /v1/me`	Current principal/scopes
Organizations	`POST /v1/organizations`	Create organization
Projects	`POST /v1/projects`	Create project
Dataset registry	`POST /v1/datasets`	Create logical dataset
Versions	`GET`	List versions

Area	Method and route	Operation
	/v1/datasets/{id}/versions	
Manifests	GET /v1/datasets/{id}/versions/{vid}/manifest	Retrieve manifest
Imports	POST /v1/imports	Begin import/connector scan
Uploads	POST /v1/uploads	Create multipart upload session
Recipes	POST /v1/recipes	Validate and save recipe
Recipe revisions	POST /v1/recipes/{id}/revisions	Freeze revision
Estimate	POST /v1/recipes/{id}/estimate	Preview cost and capacity
Runs	POST /v1/runs	Execute recipe revision
Run state	GET /v1/runs/{id}	Progress/status/usage
Run control	POST /v1/runs/{id}/cancel	Cancel execution
Generation	POST /v1/generations	Higher-level text/image/audio job
Distillation	POST /v1/teacher-jobs	Authorized teacher data generation
Simulation	POST /v1/simulations	Launch simulation recipe
Annotation	POST /v1/annotation-tasks	Create human/auto task
Review	POST /v1/annotation-tasks/{id}/decision	Record review decision
Quality	POST /v1/evaluations	Run quality evaluation
Exports	POST /v1/exports	Package and transfer output
Compute	GET /v1/compute/pools	Capability/health inventory
Compute	POST /v1/compute/agents/registrations	Request agent registration
Webhooks	POST /v1/webhooks	Subscribe to events
Metering	GET /v1/usage	Usage and estimated charges
Audit	GET /v1/audit-events	Permission-scoped audit

Area	Method and route	Operation
		search

8.3 Example request and response

```
POST /v1/runs HTTP/1.1
Authorization: Bearer <service-token>
Idempotency-Key: robotics-2026-nov-001
Content-Type: application/json

{
  "recipe_revision_id": "rrv_01K...",
  "dataset_id": "ds_01K...",
  "budget": {"max_usd": 800},
  "notifications": {"webhook_id": "wh_01K..."}
}
```

```
{
  "run_id": "run_01K...",
  "status": "QUEUED",
  "status_url": "/v1/runs/run_01K...",
  "estimated_cost_usd": 510,
  "budget_ceiling_usd": 800,
  "request_id": "req_01K..."
}
```

The estimate is *illustrative, not a price quote*. A job must fail closed when a hard budget ceiling is reached; no worker is authorized to continue accruing cost unless the customer explicitly raises the budget.

8.4 Webhook envelope

```
{
  "specversion": "1.0",
  "id": "evt_01K...",
  "source": "/organizations/org_01K...",
  "type": "ai.datafoundry.run.succeeded.v1",
  "time": "2026-11-04T09:30:00Z",
  "subject": "runs/run_01K...",
  "data": {
    "run_id": "run_01K...",
    "dataset_version_id": "dsv_01K...",
    "manifest_hash": "sha256:..."
  }
}
```

HTTP signature headers include event timestamp and HMAC signature over exact payload bytes. Consumers must deduplicate using event ID. Delivery semantics are at-least-once, not exactly-once.

8.5 SDK, CLI, and integrations

Python SDK: Typed async/sync clients, retries for safe reads, chunked upload helper, pagination iterator, `wait_for_run`, streaming export, local schema validation, sample loaders, notebook examples, optional integrations with Hugging Face Datasets and PyTorch dataloaders.

TypeScript SDK: Node/browser-safe calls, restricted client-side auth flows, upload helpers, webhook verifier and typed API methods.

CLI: `foundry login`, `dataset create`, `dataset import`, `recipe validate`, `run submit`, `run watch`, `run cancel`, `dataset publish`, `dataset export`, `compute pools`.

External integrations: Hugging Face repositories, S3/GCS/Azure Blob, PostgreSQL, Parquet, OpenLineage listeners, W&B/MLflow evaluation metadata, and model training pipelines. Providers are adapter plugins with explicit supported operations and tested schemas.

9. Generation and teacher-model studio

9.1 Workflows

A teacher job transforms a **task specification** into a versioned candidate dataset: seed tasks -> diversity planner -> model request -> structured parser -> verification -> duplication filter -> policy/safety filter -> sampling/review -> publish.

Support instruction tuning, structured extraction, classification, task-specific Q&A, code with runnable unit tests, preference pairs (candidate A/B with judge criteria), permitted demonstrations, and carefully verified multimodal descriptions.

9.2 Verification hierarchy

- Highest confidence: formally checkable/math proofs where possible; executable tests, deterministic schema validators, exact known-answer programs.
- Intermediate: independent retrieval against approved evidence, tool-grounded verification, multiple independent judges, human adjudication.
- Lowest confidence: a teacher model's own self-reported confidence or agreement with itself. Never count this as ground truth.

Track refusal rates, content-filter reasons, per-category acceptance, diversity, answer correctness on verified subsets, average cost/accepted sample, and downstream benchmark contribution.

9.3 Provider-use and distillation restrictions

Critical release gate: No provider connector may assert that output is permitted for training a competing model without a current legal/use-rights check. Restrictions differ by provider, agreement, location, and intended use. For example, OpenAI's current business terms restrict use of outputs to develop competing AI models, with documented exceptions; forthcoming policy clarifications become effective 20 November 2026. The platform must represent this explicitly as `source_terms_version`, `permitted_use`, `approval`, and `policy_decision` rather than assuming every API output is trainable.

Default to *licensed open-weight models* or providers with express documented permission for the intended downstream training. Allow customers to attach evidence of rights and restrict export or downstream use when rights are incomplete. Prompt/response retention must respect customer privacy and provider agreements. Avoid storing private chain-of-thought; use independently generated explanations and externally verified answer artifacts where appropriate.

10. Multimodal synthetic generation

10.1 Text

Task generator, prompts/templates, style and language controls, independent validators, difficulty balancing, answer format constraints, privacy filters, train/validation/test split rules, embedding-based diversity checks and accepted-sample metering.

10.2 Images

Text-to-image, image-to-image and procedural composition adapters; asset lineage, prompt and seed capture where provided; metadata for dimensions, masks, boxes, keypoints and classes; duplicate checks; aesthetic and task-specific quality measures; caption-label consistency; review interface. Keep generated synthetic labels distinct from human-verified labels.

10.3 Audio and speech

TTS and audio generation providers; transcript alignment; speaker metadata; license/voice consent checks; noise/clipping and duration metrics; anonymization for real recordings; multilingual coverage reporting. For voice cloning or speaker-identifiable output, require verified authorization and logging.

10.4 Video and time series

Scene/clip prompts, frame timestamps, audio sync, temporal labels, event boundaries, optical motion consistency, compression validation, content rights, and sampling strategies. Large media must stream through object storage and background workers.

10.5 Multimodal alignment

Every example with multiple modalities must define a **joint sample ID**, synchronization anchors, alignment version, label coordinate systems, units and sensor origin. Mismatched frame/annotation pairs fail publication tests.

11. Simulation studio: Blender, Isaac Sim, Isaac Lab

11.1 Divide responsibilities correctly

Blender: Procedural 3D assets/environments, geometry variation, materials, camera/light randomization, RGB render, custom annotations/scripts and offline preview. Export assets using validated USD/glTF pipelines when supported by the downstream engine.

Isaac Sim + Omniverse Replicator: Physically grounded simulation and synthetic perception labels; render camera sensors and annotations such as segmentation/depth/bounding boxes. Supports simulation-worker images pinned to supported drivers and RTX GPUs.

Isaac Lab: Robot learning tasks, environment parallelism, RL/imitation data, observation/action/reward trajectories and training/evaluation workflows. Some Isaac Lab modes run without Isaac Sim; full Isaac Sim rendering has different hardware requirements. Keep runtimes and capability tags separate.

Important compatibility: NVIDIA documentation distinguishes CUDA compute GPUs from RT-core renderer GPUs. An H100 can be useful for inference and many ML tasks but **must not be assumed capable of Isaac Sim RTX rendering**. Worker registration must advertise verified GPU architecture, RT-core capability, VRAM, driver, CUDA, headless rendering support, simulator version and license acceptance.

11.2 Simulation recipe specification

- Scene templates: warehouse, factory, outdoor, construction, lab, domestic and customer-imported USD scenes.
- Parameters: object spawn distributions, lighting/weather, friction/material, gravity/contacts, dynamics/noise, occlusion, motion blur, scenario constraints.
- Robot: description, articulation, end effectors, navigation or manipulation task, action space, controller version.
- Sensors: RGB, depth, segmentation, LiDAR, IMU, pose, contact and optional event camera; each with calibration and sampling rate.
- Episode: environment seed, initial state, policy/controller, termination criterion, timestep, action/reward schema, success/failure labels.
- Output: sensor media, trajectories, metadata, scene revision, labels, replay support, and conversion to target format.

11.3 Browser experience

V1/P1 starts with a parameter-driven scene form, preconfigured templates, a 3D scene preview (prefer generated low-resolution snapshots/streaming), run settings and episode viewer. Full interactive remote desktop streaming can be added later, subject to GPU/latency/cost studies. Do not try to run Isaac Sim physics and RTX rendering inside the browser.

11.4 Sim-to-real program

Import a real-world calibration/validation dataset; compute synthetic-vs-real divergence metrics for important features; define held-out real-world tests; run randomized parameter sweeps; compare transfer performance. The system can recommend additional simulations, but must not label the synthetic output as real ground truth.

12. Data intelligence, labeling, and evaluation

12.1 Ingest and curate

Data connectors support uploads, cloud buckets, dataset registries, enterprise sources and batch extraction. Processing steps: malware/sandbox scan -> rights and classification -> schema inference -> sample extraction -> integrity check -> normalizing -> exact hash dedup -> near/semantic dedup -> PII scan -> task-specific quality -> review -> split and shard -> manifest publication.

Use distributed compute only where needed; locally runnable operators should remain available for cost-effective small datasets. NeMo Curator can accelerate multimodal curation, with native plugin wrappers responsible for logs, metrics, permissions and reproducibility.

12.2 Human annotation product

Annotation project templates: text span/category, preference ranking, bounding box/polygon/keypoints, video temporal segment, speech transcript, and robotics episode review. Support assignments, gold-standard tasks, inter-annotator agreement, adjudication, audit log and reviewer analytics. Store human annotations as versioned evidence rather than overwriting model labels.

12.3 Quality gates

Each recipe declares **blocking thresholds** and **informational metrics**. Standard validation dimensions: completeness, schema validity, exact/near duplicate ratio, annotation accuracy, source-rights coverage, PII incidence, multimodal alignment, class balance, train/test contamination, bias/performance slices, simulation validity, model-impact evaluation.

A dataset version is published only when: manifest/checksum validation passes; blocking checks pass; required rights are present; no exceeded budget/approval remains open; mandatory reviewer decisions are resolved; and an auditable publication event exists.

12.4 Closed-loop improvement

Model evaluation artifacts from external training runs should map to dataset versions and benchmark versions. Use failure clusters -> data coverage hypotheses -> targeted recipe branch -> blind validation -> A/B training comparison. Freeze protected evaluation sets and avoid training on model evaluation prompts/answers. A change is beneficial only after repeated statistically meaningful evidence, not because generated sample count increased.

13. Distributed compute fabric and HPC

13.1 Hardware capability contract

Worker descriptors must include cpu_arch, cores, ram_gb, gpu_vendor, gpu_model, gpu_count, vram_gb, driver, cuda/rocm/tpu_runtime, rtx_rendering, available_software, container_runtime, network_policy, region, max_job_hours, and per-resource pricing/quotas. A workload specifies hard requirements and soft preferences; scheduler rejects incompatible matches before executing.

Workload	Preferred workers	Explicit exclusions / notes
Text parsing, validation and metadata	CPU	Avoid expensive GPU allocation
Batch teacher inference	GPU or provider API	Provider license/quotas, model-server support
Image generation	Compatible GPU model server or provider	VRAM and driver/runtime compatibility
Blender rendering	CPU or supported GPU render workers	Renderer backend matters
Isaac Sim RTX perception	Supported RTX GPU/driver	Compute-only GPUs such as H100 are not substitutes
Isaac Lab headless learning	Supported CUDA GPU/physics runtime	Exact simulator/renderer mode matters
Large-scale curation	Ray CPU/GPU cluster	Size partitions and data locality
HPC customer jobs	Slurm-backed cluster	Queue/accounting and egress policies

13.2 Scheduling policy

Admission pipeline: validate policy -> estimate reserved cost -> select eligible pool -> enforce tenant quota -> reserve spend limit -> stage inputs -> dispatch -> heartbeat -> checkpoint -> collect result and billable usage -> release reservation. Selection weights favor correctness, locality, reliable queue time, projected cost and hardware suitability. Support priority classes, preemption only when safe, retry with placement constraints, queue fairness, cancellation and orphan-job reconciliation.

13.3 Private agent architecture

Agent establishes outbound mTLS session, registers attested version/capabilities, polls or streams signed job assignments, fetches narrowly scoped credentials, runs sandboxed images, signs result hashes and emits

heartbeat/usage. Admin can revoke agent and rotate keys. All job instructions validate schema and allowlisted image digests. Avoid exporting raw private data except to user-approved destinations.

13.4 Cost control

Preflight estimator for API-token consumption, GPU-hours, CPU-hours, storage, data egress, simulator render duration and labeling review. Record estimate uncertainty; reserve budget; update forecast continuously; halt on hard ceiling; require explicit user approval for increases. Show **cost per validated sample/episode**, not just compute hours. Attribute usage to workspace/project/job and reconcile provider billing.

14. Security, privacy, legal and governance

14.1 Threat model and boundaries

Principal risks: cross-tenant IDOR/BOLA, exposed secrets, malicious uploaded documents or code, SSRF via connector URLs, harmful generation abuse, command execution in container workers, unauthorized distillation, copyrighted/private datasets without rights, data exfiltration, prompt injection through training materials, runaway compute cost, poisoned datasets and supply-chain compromise.

Mitigations: centralized authorization checks + tenant-scoped SQL queries; separate account and service roles; HTTPS/mTLS; vaulted secrets; KMS envelope encryption; egress allowlists and URL validation; malware scanning; sandboxed untrusted execution; signed worker images and SBOM; provider terms registry; source licenses/consent; spend ceilings; two-person approval for sensitive jobs; retention/deletion policy; immutable audit trails; incident response.

14.2 Access control model

Default roles: Org Owner, Billing Admin, Platform Admin, Dataset Editor, Annotator, Reviewer, Compute Operator, Read-only Analyst, Service Account. Use granular permissions (`dataset:read`, `dataset:publish`, `run:create`, `compute:register`, `provider:use`, `billing:view`, `audit:read`, etc.). Require explicit approval to publish to public registry or dispatch jobs to third-party provider APIs.

14.3 Privacy by design

Minimize stored personal data; redact/transform PII with review logs; distinguish user-uploaded sources from generated outputs; record data subject rights and deletion requests; respect region allowlists; make retention configurable. Deleted data may require cryptographic erasure and background purge of derived versions, backups, embeddings, and cached workers subject to lawful retention obligations. Document exceptions.

14.4 Governance gates

A data-use policy is machine-readable: `origin`, `license`, `contract_version`, `allowed_processing`, `allowed_training_use`, `export_destinations`, `retention_days`, `needs_human_approval`, and `policy_status`. Policy is evaluated at import, generation request, publication, and export. A customer declaration alone is not sufficient evidence for restricted third-party API training use; permit exception handling with review.

14.5 Compliance readiness

Build controls and evidence aligned to prospective SOC 2 audits, GDPR obligations where applicable, India DPDP Act obligations where applicable, and organizational procurement requirements. Do not claim certification or legal compliance merely by implementing controls. Obtain legal review before broad public rollout or sensitive-domain support.

15. Reliability, observability, and operations

15.1 Reliability design

Durable workflow histories, idempotent steps, transactional outbox for events, resumable multipart transfer, checksum verification, dead-letter queues, poison-message isolation, heartbeat lease expiry, checkpoint-aware retry and explicit FAILED/BLOCKED/CANCELLED terminal states. Separate the availability SLO for the control API from compute turnaround and third-party API success.

15.2 Initial targets (to validate)

Indicator	Proposed target	Measurement
Control-plane API uptime	99.9% monthly	Availability SLI excluding planned policy-defined maintenance
Metadata GET latency	p95 < 300 ms at stated test load	Server-side tracing, region-anchored
Accepted job durability	No silently lost accepted runs	Persist before responding; recovery test
Webhook semantics	At least once, signed, deduplicatable	Redelivery and replay tests
Dataset integrity	100% published artifacts manifest-verified	Hash audit on publish and sampled recheck
Metadata recovery point	RPO <= 15 min target	Backup PITR restore exercise
Incident recovery	RTO <= 4 h target	Staging failover/restore drill
Publish-policy correctness	No publish when mandatory gate fails	Security and e2e tests

These are *design targets*, not SLAs offered to customers until measured under production conditions. Compute throughput SLOs must be engine-, hardware-, and region-specific.

15.3 Observability dashboards

Track API error/latency, worker queue depth, job failure and retry causes, GPU/CPU/memory utilization, simulator render speed, accepted samples per hour, quality rejection by reason, average queue time, data egress, provider quota errors, token usage, budget burn, signed URL issuance, annotation throughput, security events and cost per usable dataset unit.

15.4 On-call readiness

Create runbooks for queue backlog, stalled worker, failed simulator deployment, missing artifacts, unauthorized access, credential leak, billing discrepancy, failed webhook, quota exhaustion, misconfigured region, corrupted manifest and disaster recovery. Every run has a correlation ID and human-readable error category.

16. User experience and navigation

Primary navigation: Overview | Projects | Datasets | Create Data | Teacher Studio | Simulation Studio | Labeling | Workflows | Evaluations | Compute | Usage & Billing | Settings | Developer Docs.

Dataset page: Metadata/versions; sample explorer; annotations; lineage; quality reports; generation history; evaluations; export; policy and license; collaboration comments.

Workflow builder: Canvas DAG with typed nodes and strict contracts; YAML/code view; parameter editor; tests; price estimate; validation; approval; run button; live execution view; replay and retry controls. Never hide changes between visual canvas and recipe source.

Simulation studio: Scenario template gallery, USD asset import, robot/sensor controls, randomization panel, compute compatibility check, preview/episode viewer, annotation overlays, data export.

Developer portal: API keys/scopes, organizations, documentation, interactive OpenAPI explorer, webhooks, SDK snippets, quota status, logs, sandbox/testing credentials.

Accessibility: Keyboard navigation, labeled controls, contrast, large dataset virtualized tables, actionable error messages, responsive desktop/tablet views; simulation editor may require desktop for full functionality.

17. Deployment strategy and environments

- **Local development:** Docker Compose for PostgreSQL, S3-compatible MinIO, API, workflow services, local worker, and dashboard; fixtures and mocked billable providers. Avoid pretending mock generators represent real model outputs.
- **Staging:** Dedicated accounts/secrets, production-like policies, shared test GPU pool, synthetic test data only, continuous end-to-end testing, load and security exercises.
- **Production SaaS:** Private network workloads, managed Postgres/backup, object storage, ingress/WAF, environment-specific identity, monitoring and centralized audit. Control services run redundant replicas; workers scale elastically.
- **Private compute:** Outbound authenticated agent installed on customer infrastructure; optional customer-controlled input/output buckets.
- **Enterprise private control plane:** P2 if justified by demand and compliance; includes update channel, observability export and licensing.

Infrastructure as code covers environment bootstrapping, IAM, buckets, queues, clusters, secret policies, dashboards, network rules, backups and regional configuration. Separate staging and production secrets, and disallow direct production deployment from unreviewed branches.

18. Testing and quality strategy

18.1 Test pyramid

Unit tests: Recipe parser/DAG validator; authorization; permission policy; budget calculations; sample validators; cost meters; canonical hashing; idempotency; export format conversions.

Contract tests: OpenAPI schemas; SDK generated clients; provider adapters; executor plugin schema; webhook signature and retry behavior; manifest schema migrations.

Integration tests: PostgreSQL+object storage+durable workflow+local workers; multipart upload; policy-enforced publish; cancellation; checkpoint retries; worker liveness; data export.

Infrastructure tests: Run small Ray worker jobs, a pinned Blender scene render, Isaac-compatible hardware smoke tests, Slurm queue simulation and real test queue, GPU capability rejection, node drain behavior. Do not mark an integration complete without a test on compatible hardware.

Security tests: Cross-organization resource ID fuzzing, API key scopes, expired/forged URLs, SSRF, privilege escalation, sandbox escape defense review, secret leakage checks, dependency scans and abuse throttles.

Chaos/recovery: Kill worker mid-step; corrupt one artifact; drop callback; replay webhook; exceed budget; disable provider; lose cluster connectivity; recover database from backup. Assert no invisible billing or data corruption.

18.2 Demo scenarios (release gates)

1. **Language path:** Create dataset via API -> licensed teacher adapter -> schema validation -> dedupe -> human review -> published JSONL/Parquet -> verified manifest -> signed export -> webhook.
2. **Vision path:** Upload image archive -> scan/decode -> AI suggestions -> human corrections -> COCO export -> version compare.
3. **Simulation path:** Render one parameterized Blender scene -> annotations -> manifest; later Isaac Sim RGB/depth/segmentation with RTX worker and LeRobot episode export.
4. **HPC path:** Register private Slurm agent -> eligible job placement -> heartbeat and accounting -> cancel/retry -> data remains within allowed residency policy.
5. **Failure path:** Same idempotency key twice -> one billable job; worker crash -> safe resume; invalid rights -> publish/export denied.

19. Delivery roadmap: milestones and dependencies

Estimate basis: A focused cross-functional team (for example 5-8 engineers and fractional design/security/DevOps), existing managed services, limited initial connector surface. Estimates vary substantially with team size, prior code, provider access and simulator hardware. Dates are relative, not commitments.

Milestone	Indicative weeks	Deliverables	Acceptance gate
M0 — Architecture & repository	1-2	ADRs, monorepo, design system, threat model, schemas, local Compose, CI	Everyone can start the stack locally and tests run
M1 — API + registry	3-5	IAM, datasets/versions, signed uploads, metadata, audit, OpenAPI	Tenant isolation and upload-to-version API test
M2 — Durable execution	6-8	Recipes, engine adapters, workflow runs, retries, cost ceilings, events	Kill/retry/ idempotency/policy tests pass
M3 — First production data path	9-11	Authorized teacher or synthetic import, validation, review, exports, webhooks, Python SDK	Full API-to-dataset vertical slice works
M4 — Curate and visualize	12-15	Image annotation, quality dashboards, semantic indexing, enhanced studio	Vision sample import/label/export validated
M5 — Simulation and distributed compute	16-21	Blender + compatible RTX Isaac integration, Ray, Kueue, Slurm agent	Verified simulator run on real compatible hardware
M6 — Reliability and launch readiness	22-26	Hardening, pentest fixes, DR rehearsals, billing reconciliation, support runbooks	Launch checklist and approval gate satisfied

Parallel workstreams: (A) APIs + schema, (B) dashboard + UX, (C) workflow + worker runtime, (D) data/ML adapters, (E) infrastructure + security. Avoid starting M5 deep integration before M2 worker contracts stabilize. If the team is smaller, preserve acceptance gates and extend the schedule.

19.1 First 30-day engineering backlog

Week 1: Repository and CODEOWNERS; ADR-001 control/data planes; dataset and recipe JSON Schema; PostgreSQL migrations; Docker Compose; AuthN/AuthZ foundation; landing dashboard; CI baseline.

Week 2: Dataset CRUD, multipart upload and integrity checks, manifest builder, tenant isolation tests, recipe validation endpoint, storage abstraction, sample dataset fixtures.

Week 3: Temporal workflow and local worker, run API/state transitions, job logs, retry policy, cost estimate/reservation, execution tracing, stubbed provider adapter explicitly labeled as fixture.

Week 4: One legitimate end-to-end data path using an approved real provider or permitted local generator; quality checks, publish/export, signed webhooks, Python SDK smoke test; launch-demo script and blocking defect fixes.

19.2 Definition of done for every feature

- Requirements and acceptance criteria tracked in issue/PR.
- Typed API schema, authorization rules and migration impact documented.
- Happy path + failure path tested; tenant boundary and cost policy covered.
- Logs/traces/metrics wired; billing usage consistent; no secret in output.
- User-facing docs and example code updated.
- Feature flag or safe rollout strategy defined; production usage not claimed until verified.

20. Staffing, ownership and operating model

Workstream	Primary owner	Partner
Product / prioritization	Product lead	Research lead
UI and workflow studio	Frontend engineer	Product designer
API and tenancy	Backend/platform lead	Security engineer
Recipe execution and workers	Distributed systems engineer	MLOps engineer
Data quality and adapters	ML/data engineer	Annotation lead
Simulation engines	Robotics/simulation engineer	GPU/HPC engineer
Compute clusters / IaC	Infrastructure/SRE engineer	Security engineer
Policy and vendor terms	Legal/privacy reviewer	Product lead
Reliability/incident management	SRE on-call	Engineering lead

Recommended ceremonies: weekly architecture/risks review, prioritized two-week iterations, PR checklists, demo of one end-to-end workflow per milestone, monthly cost/security review, and a documented decision log. Require design review for any new engine contract or data schema because those become long-lived ecosystem interfaces.

21. Cost model and commercial product design

21.1 Cost drivers

Total cost per run = provider inference and generation fees + CPU/GPU/accelerator runtime + storage and requests + network egress + human review + supporting orchestration/observability overhead + amortized idle/queue capacity. Forecast in low/base/high scenarios and display uncertainty.

Do not invent list prices; price tables are dynamic, region-specific, and dependent on contracts. Pull provider rates through a controlled pricing catalog with versioned effective dates and verified source. Store actual usage separately from estimates. Reconcile daily against provider invoices where supported.

21.2 Proposed packaging

- **Developer:** Self-service projects, limited storage/compute, API keys, community support, strong free or capped trial safety limits.
- **Team:** Shared workspaces, expanded jobs, annotation collaboration, dataset review, metered usage and priority queues.
- **Enterprise:** SSO/SCIM, private compute, customer-owned keys, audit export, residency contracts, governance and custom support.

Bill on a transparent metered basis: platform subscription + pass-through or marked-up compute/API usage, storage, and human review where applicable. Decide whether customers bring their own model/provider keys; if so, use vaulted references and separate billing semantics.

22. Risk register and mitigations

Risk	Severity	Mitigation / evidence required
Unauthorized teacher-model distillation	Critical	Terms registry, permitted-use policy and legal approval gate
GPU/simulator incompatibility	High	Capability-based placement and real hardware certification
Data leakage across tenants	Critical	End-to-end authorization, isolated storage, adversarial tests
Unbounded model/API or GPU spend	Critical	Hard budget ceilings, reservations, metering, cancellation
Low-quality synthetic dataset harms model	High	Independent verification, human review, benchmark A/B
Train/test contamination	High	Protected eval sets, contamination scanning, provenance
Incomplete dataset lineage	High	Immutable manifest and blocking publication checks
Vendor lock-in	Medium	Adapter contracts, standards, portable object-store formats

Risk	Severity	Mitigation / evidence required
Unrealistic scope/velocity	High	Vertical slice first; hard release gates; P1/P2 deferral
Workflow/event inconsistency	High	Transactional outbox, idempotency, reconciliation tools
HPC agent compromise	Critical	Outbound mTLS, sandboxing, signed work units, revocation
Massive data egress fees	High	Data-local jobs, regional policy, cost-aware placement
Licensing of generated images/3D assets	High	Source licenses, consent/rights fields, export policies
Regulatory or procurement gap	High	Control evidence, legal review, staged beta

23. Decision log (recommended defaults)

ID	Decision	Default	Revisit when
ADR-001	Control plane / data plane	Separate logical layers and worker identities	Private installation requests increase
ADR-002	Deployment shape	Modular monolith + workers	Throughput/team ownership forces split
ADR-003	Workflow engine	Temporal	Ops burden warrants managed alternative
ADR-004	Metadata store	PostgreSQL	Analytical query demands justify new store
ADR-005	Artifact storage	S3-compatible object store	Customer storage interoperability requires adapter
ADR-006	Engine contract	Versioned recipe adapter + capability registry	Plugin ecosystem matures
ADR-007	Tenant model	Shared control plane, isolated data execution	Enterprise dedicated tenancy demanded
ADR-008	Simulation	Blender first; Isaac on certified RTX pool	Real robotics beta use cases
ADR-009	API	REST/OpenAPI first; async operations	Streaming/ interactivity warrants gRPC

ID	Decision	Default	Revisit when
ADR-010	Data governance	Fail closed for restricted sources	Explicit reviewed exception

24. Production launch checklist

Security: [ ] Threat model reviewed. [ ] Per-tenant authorization tests. [ ] Secrets manager and key rotation. [ ] Worker sandbox review. [ ] Audit log immutability. [ ] External pentest or independent review. [ ] Vendor contract and data-use checks. [ ] PII/retention process.

Reliability: [ ] Backup and PITR restore tested. [ ] Recovery exercise. [ ] Jobs persist through worker outage. [ ] Dead-letter/retry verified. [ ] Idempotent billable operations. [ ] Webhook signature replay test. [ ] Incident on-call and runbooks.

Data quality: [ ] Manifest hashes checked. [ ] Dataset schema tested. [ ] Annotation accuracy assessed. [ ] Quality gates are blocking. [ ] Source rights established. [ ] Split leakage checked. [ ] Evaluation summary attached.

Developer experience: [ ] OpenAPI published. [ ] Python SDK released. [ ] Error model stable. [ ] Example notebook works. [ ] Signed upload/download examples. [ ] CLI and TS SDK release status clearly marked. [ ] Rate limits documented.

Infrastructure: [ ] Version-pinned containers. [ ] Production IaC reviewed. [ ] Staging parity. [ ] GPU compatibility tests. [ ] Quota/budget alerts. [ ] Cost reconciliation. [ ] Private agent revocation tested. [ ] Sim and HPC integrations explicitly marked available/unavailable per region.

Business readiness: [ ] Terms/privacy drafted. [ ] Support and incident escalation. [ ] Subscription and metered billing reviewed. [ ] Test customer onboarding completed. [ ] Launch permissions obtained. [ ] No unapproved paid infrastructure.

25. Acceptance criteria for a handoff to ChatGPT Work

This section is the implementation contract for an agent working on the project. The agent must **create runnable code**, not stop at a mockup or architecture-only explanation.

25.1 Expected repository layout

```
ai-data-foundry/
  apps/
    web/                # Next.js studio
    api/                # FastAPI control plane
  services/
    workflow-worker/    # Temporal workflows/activities
    local-executor/     # local CPU processing for V1
    compute-agent/      # P1 remote agent
  packages/
    schemas/            # JSON Schemas/OpenAPI/source contracts
    python-sdk/         # typed client
    ts-sdk/             # later release
    cli/                # later release
    recipes/            # validation / compiler
    engine-sdk/         # provider/engine plugin interface
```

```
integrations/
  teacher-openweight/    # only licensed/approved use
  blender/               # procedural scene runner
  isaacsim/              # version-pinned simulator adapter
  isaaclab/              # learning/episode adapter
  slurm/                 # HPC scheduler adapter
infra/
  compose/ terraform/ kubernetes/
tests/
  unit/ integration/ contracts/ e2e/ security/
docs/
  architecture/ api/ operations/ runbooks/ adr/
scripts/
  dev-up.sh demo-e2e.sh seed-fixtures.sh
.github/workflows/
README.md
```

25.2 Required deliverables from implementation agent

1. A runnable local stack with setup/seed commands and no undisclosed credentials.
2. Working API endpoints with OpenAPI 3.1 and authenticated tenant boundaries.
3. Durable recipe job submission, validation, state transitions and real local execution.
4. Actual dataset upload, checksum verification, sample processing, quality report, version manifest, export and signed webhook.
5. Python SDK and minimal API usage example that succeed against local deployment.
6. Clean studio UI: datasets, recipes, runs, quality, exports, compute and settings, with honest feature availability.
7. Automated tests covering a normal path, failure/retry, cross-tenant denial, policy blocking, and idempotency.
8. Docker/development environment, deployment infrastructure skeleton, CI validation and migrations.
9. Documentation, architecture decisions, known gaps, cost assumptions and next prioritized issues.
10. No fake inference, simulated simulator output, fake GPU availability, or unapproved production deployment represented as real functionality.

25.3 Agent execution sequence

Step A — Inspect environment. Discover repository, constraints, credentials availability (without printing secrets), connected services, existing code, and actual permission to push/deploy. Record baseline.

Step B — Implement vertical slice. Build local stack, schema, auth, dataset API, recipe, workflow, local data processing engine, validations, manifest, export and webhook. Demonstrate one complete run with tests.

Step C — Expand interfaces. Add dashboard with functional state from API, Python SDK, sample notebooks and thorough docs. Avoid placeholder success messages.

Step D — Integrate engines. Add adapters one by one under feature flags. For external model providers, use approved terms and credentials; for GPU simulation, report required hardware and provide integration tests only if compatible GPUs are actually available.

Step E — Harden. Add metering and cost ceilings, audit, policy engine, error reporting, rate limiting, recovery, security tests and deployment previews.

Step F — Validate before declaring done. Run tests and smoke workflows, record evidence and errors, identify blocked items, produce a prioritized backlog and request explicit approval before paid resources or production deployment.

Work-mode master instruction (paste as a task)

> *Implement AI Data Foundry using this master plan as the source of truth. Build an actual API-first training data platform, starting with a complete local end-to-end dataset lifecycle. Use Next.js + TypeScript, FastAPI, PostgreSQL, S3-compatible object storage, durable workflows, typed schema and a Python SDK. Keep control plane and data plane distinct. Build clean, production-oriented code with authentication, tenant boundaries, quality gates, immutable manifests, audits, cost ceilings, reproducible examples, tests, CI, documentation and an excellent UI. Add authorized teacher generation, multimodal curation, Blender, Isaac Sim/Isaac Lab and distributed hardware adapters incrementally, reporting any inaccessible provider or hardware integration honestly. Never fabricate execution outputs or claim deployed features that were not verified. Do not spend or deploy to production without approval. Show runnable results, tests, commit/PR artifacts when possible, and the remaining issue list.*

26. Open questions to resolve during build—not blockers to starting

1. Product name and ownership/brand: use **AI Data Foundry** as working name; rename at launch if needed.
2. First customer segment: default to AI research teams and small platform teams; postpone all-in-one enterprise claims until validated.
3. First authorized teacher provider: use a licensed local open-weight model or approved API; do not assume commercial API output training rights.
4. Cloud: choose a primary hosted environment based on existing contracts and cost; keep infrastructure portable.
5. Initial input connectors: local upload, S3-compatible bucket, Hugging Face dataset ID, and one structured file import.
6. Preferred workspace identity provider and billing system: select after vendor/security review.
7. GPU access: start with API/CPU demo and Blender-compatible worker; add Isaac when certified RTX hardware is accessible.
8. Compliance regions and storage residency: default to one selected supported region; extend only when requested and tested.
9. License of the Foundry codebase: decide before releasing any source publicly.
10. Pricing levels: model with usage units first; publish only after measured workload costs.

27. Reference standards and verification sources

These references were checked for this planning draft on **4 November 2026**. They support tool selection and constraints; the implementation should verify current versions and contracts again before dependency pinning or production rollout.

1. NVIDIA Isaac Lab installation and compatibility — https://docs.nvidia.com/isaaclab/isaac-lab/installation
2. NVIDIA Omniverse Replicator synthetic data framework — https://docs.omniverse.nvidia.com/extensions/latest/ext_replicator.html
3. NVIDIA clarification on RT-core requirements vs H100 for Isaac Sim — https://docs.nvidia.com/halos/outside-in-safety/getting-started/prerequisites
4. MLCommons Croissant 1.1 dataset metadata specification — https://docs.mlcommons.org/croissant/docs/croissant-spec-1.1.html
5. Hugging Face LeRobotDataset v3.0 — https://huggingface.co/docs/lerobot/lerobot-dataset-v3
6. NVIDIA NeMo Curator — https://docs.nvidia.com/nemo/curator/about

7. Ray Data documentation — https://docs.ray.io/en/latest/data/data.html
8. Kubernetes Kueue overview — https://kueue.sigs.k8s.io/docs/overview/
9. Temporal durable AI workflows — https://docs.temporal.io/ai
10. OpenLineage specification — https://github.com/OpenLineage/OpenLineage/blob/main/spec/OpenLineage.md
11. CloudEvents JSON event format — https://github.com/cloudevents/spec/blob/main/cloudevents/formats/json-format.md
12. OWASP API Security Top Ten — https://devguide.owasp.org/en/07-training-education/07-api-top-ten/
13. OpenAI Business Terms (use-of-output restrictions) — https://openai.com/policies/may-2025-business-terms/
14. OpenAI Usage Policy changes effective 20 November 2026 — https://help.openai.com/en/articles/12092907-were-updating-our-usage-policies

---

Final principle: Build a platform where every training-data artifact is *permissioned, reproducible to the maximum feasible extent, measurable, versioned, programmable, and generated on compatible compute*. The first victory is one honest end-to-end dataset workflow; the long-term advantage is a universal data production ecosystem that can continually improve AI models using governed feedback.

V2 IMPLEMENTATION & DESIGN ADDENDUM

28. Version 2 mandate: Vercel-inspired, functional by design

Status: Approved design direction for implementation planning, not evidence of a running product. **Supersedes any earlier ambiguous UI or integration requirement.** Use the original Sections 1-27 as the baseline; this addendum makes the specification actionable for Codex.

28.1 Product principle

The application must *feel as deliberate, fast, legible, and dependable as a top-tier developer cloud console*. The visual reference is Vercel's Geist design language (structure, typography, monochrome surfaces, grid, code-first clarity, and refined interactions), **not** an instruction to copy Vercel branding, source code, trademarks, illustrations, or trade dress pixel for pixel. Our product must have its own name, visual identity, and independently implemented design system.

Non-negotiable: Nothing presented as functional may be fake. A button must navigate, call an implemented API, open a meaningful dialog, change persisted state, or be visibly disabled with an explanation. No random data masquerading as live usage. No frontend-only success messages when backend actions failed. Unsupported connectors must say Coming later / Requires configuration, not Connected. Every page needs honest empty, loading, error, permission-denied, and degraded-service states.

28.2 Vertical slices, not mockups

Use **one feature end to end at a time**: persisted database model -> authenticated API endpoint -> background worker -> stored artifact/observability -> web experience -> SDK -> tests -> docs. Screenshots and visual QA are required, but a beautiful screenshot is not proof that a feature works. A feature is complete only when the user journey runs using real services in a reproducible local environment.

28.3 Scope protection

The full product includes the twelve engines already described. V1 must demonstrate the platform contract on CPU and optionally through one authorized, configured API provider. Advanced video, Isaac Sim/Isaac Lab, multi-cloud

HPC, custom accelerator drivers, browser 3D editing, and full enterprise integrations are **phased integrations**. Do not simulate paid GPU jobs or label placeholders as production. All connectors expose capability, configuration, dependency, availability, permission, and health state.

29. Product information architecture and screen-by-screen design

29.1 Left navigation

Persistent workspace-aware navigation, collapsed sidebar on narrow screens, proper keyboard focus and semantics. Proposed IA:

Group	Destination	Must be functional
HOME	Overview	Organization summary, recent runs, failures, usage, setup checklist
DATA	Datasets	Browse/search/filter/ create/import; access-controlled listing
DATA	Dataset detail	Versions, sample preview, manifests, quality, lineage, exports, permissions
BUILD	Generation Studio	Text, image, audio, video requests through available generators
BUILD	Teacher Studio	Authorized model endpoints, structured output, independent verification
BUILD	Simulation Studio	Scene recipes, engine compatibility, job setup, asset previews, episode results
BUILD	Annotation Studio	Projects, queues, labeling, adjudication, quality and export
BUILD	Workflows	DAG recipe editor, schedules, revisions, run history
BUILD	Data Intelligence	Cleaning, quality profiles, dedupe, search, distribution reports
COMPUTE	Compute pools	Agents, accelerators, capacity, capabilities, prices, quotas, job placement
COMPUTE	Jobs & Runs	Queue/status/retries/ cancel, logs, checkpoints, cost, trace and outputs
MEASURE	Evaluation Lab	Dataset metrics, compare versions, benchmark results, feedback loop
MEASURE	Usage & Billing	Spending, budgets, meters, invoices if configured; clear

Group	Destination	Must be functional
		unbilled state
DEVELOPERS	API keys	Scoped credentials, one-time secret reveal, rotation, audit
DEVELOPERS	Webhooks	Endpoints, signing secrets, delivery history, replay
DEVELOPERS	API Explorer	OpenAPI-generated explorer, snippets, code examples, SDK links
SETTINGS	Connections	AI providers, buckets, Git repositories, storage, compute, test connection
SETTINGS	Members & roles	Invite, revoke, role assignment, service accounts
SETTINGS	Policies	Retention, PII, source rights, approvals, region and export policy
SETTINGS	Audit log	Query actor, action, target, IP/time, correlation IDs, export

Add a global command menu (Ctrl/Cmd+K), omnibox search, organization/project switcher, breadcrumbs, keyboard shortcuts, a notifications center, persistent help/docs, and mobile-friendly navigation. Every global affordance needs an actual behavior and tests.

29.2 Five critical routes first

1. /overview — real organization summary and onboarding states; zero values display as zero, not fabricated KPIs.
2. /datasets and /datasets/[id] — create, import, inspect, compare, publish, export; real upload status and quotas.
3. /workflows/new and /workflows/[id] — declarative recipe form with source/transform/validate/export steps; form builder first, visual DAG second.
4. /runs and /runs/[id] — honest run state, attempts, progress, error, retry/abort actions, downloadable log and trace IDs.
5. /settings/api-keys — scoped key creation/revoke and authenticated quick-start example.

Implement these before filling out the rest of the navigation. Sidebar destinations without functionality must either be hidden by flags or clearly disabled.

29.3 Primary interaction patterns

- Multi-step **Create dataset** flow: choose modality -> name/schema -> ingest/generate -> preview requirements -> quote cost -> review source-use policy -> create -> monitor -> inspect version.
- **Build pipeline**: choose an ingestion/generation node, validation and output; validate DAG at edit time; display generated DSL; confirm credits/budget; launch; show logs and cost.
- **Teacher synthesis**: choose permitted provider/teacher; present license warning and model-use permissions; prompt/schema; sample generation; independent verification; approval; run.
- **Simulation**: pick Blender/Isaac engine; display capability requirements, asset licenses and worker availability; edit recipe parameters; preview locally/remotely if available; run; inspect synchronized frames/labels.

- **Dataset review**: paginate and lazily load samples, annotate, comment, resolve disagreements, export accepted artifacts.
- **Failure recovery**: show root cause with retry classification, link to logs, budget consequences, checkpoints and retry eligibility.

29.4 Functional UI contract

Each route is defined with route ID, query API, mutations, required scopes, empty state, loading state, error state, disabled state, telemetry, a11y tests, and E2E flow. A page cannot merge without filling this contract. Interactive graph nodes, filters, sorts, pagination, file-drop zones, dataset cards, and keyboard commands must have tests; all metadata must come from real API responses or explicitly marked fixtures in development mode.

30. Visual system: Geist-inspired, original identity

30.1 Art direction

- Use a refined neutral palette, high contrast and restrained accents; bright colors are for meaning (success, failure, warning, active selection), not decoration.
- Clean desktop console, sophisticated narrow-width layouts, generous whitespace but dense enough for data engineers. Prefer a 12-column layout, consistent container widths, and an 8px spacing scale.
- Headings in Geist Sans or legally available equivalent; code/logs/IDs in Geist Mono or equivalent. Pin font packages and license compliance. Do not bundle font files in handoff documents.
- Minimal black/white landing, subtle borders/dividers, 1px hairlines, soft shadows only on overlays. Square-ish cards with small tasteful radius. Avoid gradients, glassmorphism, exaggerated animations, or arbitrary colors.
- Light and dark themes with persisted preference; system default, accessible contrast, reduced-motion behavior.
- Icons from a coherent set (Lucide or an appropriately licensed set); real tables, charts, terminal logs, graph edges, image previews and dataset samples take priority over decoration.
- Render server-owned data in server components where practical; client components only for interactivity. Provide skeletons that match the final layout and prevent hydration flicker.

30.2 Initial tokens (proposal, adjust with visual QA)

Token	Light	Dark	Purpose
Canvas	#FFFFFF	#0A0A0A	App background
Surface	#FAFAFA	#111111	Subtle secondary surface
Elevated	#FFFFFF	#171717	Popovers and modals
Text primary	#171717	#EDEDED	Main copy
Text muted	#666666	#A1A1A1	Supporting copy
Border	#EAEAEA	#333333	Delimiters
Focus ring	#0070F3	#3291FF	Accessible focus state
Positive	#16803C	#46A758	Success
Warning	#B76E00	#F5A524	Attention

Token	Light	Dark	Purpose
Destructive	#DC2626	#F87171	Irreversible/failure

These are *our proposed tokens*, not a claim of identical Vercel tokens. Define CSS variables and consume them through semantic Tailwind aliases; never hard-code colors in individual page components.

30.3 Component inventory

AppShell, WorkspaceSwitcher, Sidebar, Topbar, CommandMenu, Breadcrumbs, PageHeader, DataTable, Metric, StatusPill, EmptyState, ErrorState, ConfirmDialog, ActivityTimeline, ResourceSelector, QuotaBar, CostEstimate, KeyValueList, JSONViewer, CodeEditor, LogViewer, ArtifactPreview, SampleGallery, SchemaEditor, DAGCanvas, JobTimeline, FilterBuilder, UploadManager, DataCard, PermissionGate, InlineNotice, FormField, CopyableValue, Toast.

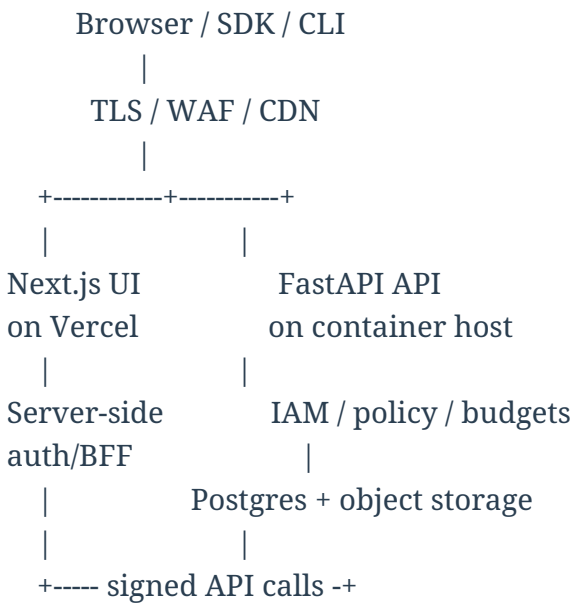
Primitive UI: Next.js App Router + React + TypeScript + Tailwind CSS + accessible Radix/shadcn-style primitives (copy to project code, customize with tokens), TanStack Query for client synchronization, TanStack Table for complex grids, React Flow for DAG canvas, Monaco or CodeMirror for DSL editing if bundle and licensing are acceptable, and Lucide. Avoid mixing four UI libraries. Storybook is optional initially; component stories/tests required at least for critical primitives.

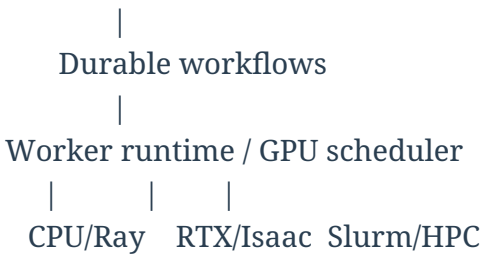
30.4 Responsive and accessibility acceptance

- Test 360px, 390px, 768px, 1024px and 1440px widths; no horizontal overflow outside intentionally scrollable data grids.
- All controls usable without a mouse; visible focus and meaningful labels; escape closes dialogs; focus returns to trigger; status updates announced where needed.
- WCAG 2.2 AA as acceptance target; automated axe checks and manual keyboard/screen reader smoke test.
- Virtualize long tables/lists; pagination server side; image/audio/video previews load progressively; support reduced motion.
- Error and success toasts include a correlation ID when actionable; never replace an error with a silent reset.

31. Runtime boundary: Vercel UI, external compute

31.1 The deployable topology





Important: Vercel is the UI hosting and edge delivery layer, **not** the default execution venue for hours-long GPU simulation or training-data generation. Even where platform functions permit relatively long durations, they have bounded execution time and lack arbitrary access to dedicated NVIDIA GPUs/Isaac runtime. Keep jobs in long-lived, isolated container/cluster workers. Frontend calls use short requests to submit and poll jobs, or subscribe to an event gateway.

31.2 Environments

Environment	Purpose	Data and billing
local	Docker Compose: API, Postgres, MinIO, workflow worker, web	Seeded synthetic fixtures, no paid integrations by default
preview	Per-PR Vercel UI; optional shared limited API/test DB	Synthetic fixtures, isolated test tenants; no production secrets
staging	Full integrated rehearsal of production topology	Dedicated DB/bucket/keys; capped resources
production	Customer-facing service with monitoring, backup, rotation and change control	Live customers; gated promotion and rollback

Vercel project connects to Git branches; preview and production use separate environment variables. API must validate trusted origins, JWT issuer/audience/scopes, and CORS according to the deployment topology. Never expose provider secrets or private agent credentials in `NEXT_PUBLIC_` variables. Direct browser-to-object-store transfers must use short-lived signed URLs and bucket CORS rules.

31.3 Deployment pipelines

PR -> lint/types/unit -> API contract check -> backend integration -> Playwright E2E -> security checks -> build -> preview -> design/functional QA -> merge -> staging migration rehearsal -> load/recovery checks -> approved production release -> synthetic production checks. Database schema migrations must be backward-compatible with the prior deployed API, and destructive migrations require a multi-release process.

32. Monorepo blueprint

```
ai-data-foundry/
  AGENTS.md                # Codex's standing instructions
  README.md                # setup, development and verified capabilities
  CODEX_START.md           # phase-by-phase launch directions
  docker-compose.yml       # local Postgres, MinIO, Temporal/worker
  Makefile                 # setup/dev/test/lint/smoke commands
  .env.example             # placeholders only: no secrets
  pnpm-workspace.yaml
```

```
turbo.json
apps/
  web/                 # Next.js App Router, design system + screens
  api/                 # FastAPI, models, routers, migrations
  docs/                # API guides + developer docs (later)
packages/
  ui/                  # reusable design system + tokens
  api-types/           # generated TS types from OpenAPI
  sdk-ts/              # TypeScript API SDK
  sdk-python/          # Python SDK with real HTTP transport
  contracts/           # JSON Schema, OpenAPI and events
  config/              # shared lint/types/tooling
workers/
  core/                # deterministic CPU jobs, quality, export
  generation/          # permitted teacher & media adapters
  blender/             # packaged headless Blender runtime
  isaac-sim/           # RTX Linux GPU runtime, independent image
  isaac-lab/           # compatible robot-learning runtime
  compute-agent/       # outbound-only signed private agent
services/
  scheduler/           # compute eligibility, placement, leases
  event-delivery/      # outbox, signed webhooks and retries
infra/
  terraform/           # versioned production infrastructure
  k8s/                 # GPU workers and Kueue profiles
  slurm/               # adapter submission/test fixtures
  vercel/              # frontend config/reference
tests/
  e2e/                 # Playwright full user journeys
  integration/         # API + DB + S3 + workers
  security/            # cross-tenant and auth negatives
  load/                # k6 or Locust benchmarks
  fixtures/            # versioned synthetic data only
docs/
  architecture/        # ADRs, threat model, diagrams
  product/             # feature matrix, acceptance criteria
  runbooks/            # incident, rollback, backup/restore
  api/                 # API versioning and webhook behavior
  designs/             # tokens, screens, interaction contract
.github/workflows/     # CI, PR validation, SAST/dependency checks
```

Do not create dozens of empty packages just to match this tree. Create each directory when its first working executable, contract, or test lands. Tests and docs may be created early to define intent; runtime modules without implementation are explicitly marked not implemented and are excluded from default UI navigation.

33. API and SDK contract: production-grade behavior

33.1 Rules

- Public base path /v1; OpenAPI 3.1 source of truth, generated TS types and Python client contracts; additive changes are compatible, breaking changes require a new version.
- Authentication: OIDC session for browser, short-lived scoped service tokens/API keys for service integration, tenant verified from authenticated principal rather than from arbitrary request body.
- Request correlation ID, RFC 7807 Problem Details errors, trace ID, consistent UTC RFC 3339 timestamps and opaque ULIDs/UUIDs.
- Cursor pagination (limit, cursor), stable ordering, filter syntax and sort whitelist; limits enforced server-side.
- Create/launch endpoints accept Idempotency-Key and canonical request hash; replay identical request yields same resource, mismatched body yields conflict.
- API returns 202 Accepted for compute-bound launches with Location run URL. Never hold a request open until a GPU job completes.
- Exact-once externally observable effects are achieved by idempotency and outbox dedupe, not by claiming distributed workers execute exactly once.
- Mutations enforce scopes and per-resource policy; archived dataset versions remain immutable and only metadata explicitly designated mutable can be edited.
- Provide OpenAPI-examples for both successful and negative responses; generated SDKs handle retries only on safe/idempotent operations.

33.2 Key API resources

Resource	Primary actions	Notes
/organizations, /projects	list/create/update	Tenant-scoped administrative permissions
/datasets	list/create/search	Schema, modality, lifecycle
/datasets/{id}/versions	list/get/publish	Immutable manifest and digest
/uploads	initiate/complete/abort	Multipart direct-to-object-store, checksum verification
/imports	create/inspect	URL allowlist/SSRF protection, sources/licenses
/recipes, /recipes/{id}/revisions	validate/create/revise	Versioned DAG and engine locks
/runs	launch/list/get/cancel/retry	Durably queued, budget approved
/runs/{id}/events	list/stream	Server-Sent Events/polling fallback, resume token
/generations, /distillations, /simulations	create and get	Convenience endpoints compile recipes to runs
/annotations/tasks, /reviews	list/assign/submit/ adjudicate	Optimistic locking, reviewer audit

Resource	Primary actions	Notes
/quality-profiles, /evaluations	manage/run/compare	Quality gate evidence
/exports	create/get/download	Signed, short-lived, policy checked
/compute/pools, /compute/agents	register/status/revoke	mTLS or equivalent workload identity
/provider-connections	configure/test/rotate	No secret values in GET response
/usage, /budgets	view/change	Append-only ledger, budget lock
/webhooks, /webhook-deliveries	subscribe/rotate/replay	Signed at-least-once delivery
/audit-logs	immutable query	Privileged, filtered, retention
/health/live, /health/ready	status	No sensitive runtime details in public response

33.3 Representative sample contracts

POST /v1/datasets: name, modality, validated schema, project ID, optional retention, policy; returns 201 plus opaque ID. POST /v1/recipes: versioned DAG with source -> processor -> validator -> publisher, engine/connector versions and resource budgets; graph validated before persistence. POST /v1/runs: {recipe_revision_id, dataset_id, request_id, max_budget_usd, compute_constraints} -> 202 and run state. POST /v1/exports: {version_id, format, destination} -> asynchronous export; download via signed URL after READY.

Events use event_id, event_type, schema_version, occurred_at, organization_id, resource_id, and data. Signed webhooks include timestamp, delivery ID and HMAC SHA-256 signature; verify on raw request bytes and bounded time window. Failed deliveries are retried with backoff and dead-letter handling; users can replay authorized deliveries.

33.4 SDK must work from a blank environment

Python: from datafoundry import DataFoundry; client.datasets.create(...), client.runs.create(...), client.runs.wait(...), client.datasets.download(...); async and sync clients where reasonable. TypeScript: import {DataFoundry} from '@datafoundry/sdk' with typed error shapes, timeout/cancellation and retry policy. CLI: datafoundry auth, datasets, runs, compute, webhooks, doctor. Do not publish package names until owned and registered; initially test workspace-local SDKs against the actual API and record supported compatibility versions.

33.5 External API usage of generation providers

A provider adapter must declare: supported modalities, input/output schemas, max batch/throughput, token/image pricing metadata, content/data retention settings, API region, terms/license flags, rate limits, timeout/retry rules, credentials, and permitted downstream training uses. No teacher-derived training output is scheduled until the provider connection records explicit authorization. Keep provider response hashes, provenance and generated output in permission-scoped storage. Reserve budget before dispatch; reconcile real provider cost afterward.

34. Persistence, consistency, and data correctness

34.1 Database essentials

Use PostgreSQL for tenants, projects, datasets, versions, recipe revisions, run state/attempts, external job references, budgets/reservations, usage, audit, webhooks, outbox and approvals. Use `organization_id` on every tenant-bound table; defense in depth via scoped repository queries and optional PostgreSQL RLS. Unique indexes enforce slugs, idempotency requests, version numbering and sample digest uniqueness within a chosen dedupe scope. Use SQL migrations (Alembic for FastAPI). Retention/deletion flows must include legal holds, delayed object deletion, and derivative index cleanup.

34.2 Data in object storage, not SQL rows

Raw and output shards live in S3-compatible storage: JSONL/Parquet for text and structured samples, references to images/audio/video, manifest `sha256`, data cards, metrics, lineage and sample indexes. Use temp staging prefixes and a two-phase publish step: validate/checksum all artifacts -> transactionally publish a manifest and immutable dataset version -> make read-only. A failed publish must leave a recoverable staging run, never a half-published version.

34.3 Version semantics

`dataset_id` is mutable logical identity. `dataset_version_id` is an immutable snapshot. `recipe_revision_id` is immutable run input; `run_id` is unique per attempt group; `attempt_id` identifies retries. Canonical manifest contains artifact digests, model versions, provider rights, data splits, sample counts, quality results, config, seeds, environment digests and `created_at`. Reproducing a *deterministic* pipeline should compare identical digests; inherently stochastic external providers must be annotated `best-effort reproducible` with captured request and actual response digests, not falsely guaranteed deterministic.

35. Production execution engine and failure design

35.1 Workflow compilation

User forms and DAG canvas both compile to the same typed recipe JSON Schema. Compiler validates graph acyclicity, declared input/output modality compatibility, permissions, provider rights, environment constraints, budget and required resources. Execution plan is frozen and hashed. Workers process idempotent steps and emit structured events. Outbox publish and webhook dispatch are transactional/compensating so subscribers never receive success before a committed version.

35.2 Worker lifecycle

`SUBMITTED -> POLICY_CHECK -> ESTIMATED -> APPROVED -> ADMITTED -> ASSIGNED -> RUNNING -> VERIFYING -> PUBLISHED` plus `BLOCKED`, `CANCELLED`, `FAILED`, `EXPIRED`, and `RETRYING`. Retry only safe stages based on recorded checkpoints; user-visible error includes code, cause, owner and suggested action. Cancellation releases reservations and attempts best-effort cleanup of engine resources; accounting still records spent resources.

35.3 Long-running jobs

Use Temporal or an equivalent verified durable orchestration engine. API enqueues jobs and responds immediately; durable worker owns state. Streaming status can use SSE from a dedicated API endpoint with polling fallback; no

browser client directly watches a GPU node. Limit concurrency per tenant and provider, reserve budgets, enforce queue fairness, and reject or defer jobs requiring unavailable hardware.

35.4 Operational tests that matter

Kill worker mid-job, restart API, turn off object storage, inject provider 429/500, use stale API key, upload invalid MIME, submit duplicate idempotency keys, cross-tenant access, exceed budget, force quality gate failure, disconnect HPC agent, invalidate webhook secret, cancel after partial generation, retry after scheduler timeout. Each scenario has a deterministic integration/E2E test or a documented controlled fault injection test.

36. Exact engine integration contracts

Engine	Output contract	Constraint	Proof of real integration
Text provider	`SampleEnvelope[]`, token usage, license/provenance, validation scores	Terms permit target training use; quota/latency	Real provider call in integration environment with redacted trace
Open-weight teacher	Checkpoint digest, tokenizer, input/output records, GPU use	License and deployment environment	Reproducible inference run with captured model hash
Image generation	Image artifact + prompt/seed/settings, labels where independently checked	Provider/asset rights, content policies	Verifiable media checksum and preview
Audio generation	Audio artifact, transcript/alignment, speaker/locale metadata	Consent/licensing; voice safeguards	Playable file and alignment quality check
Video generation	Timestamped clip + frames + captions	High cost; rights and storage	Verified timeline, duration and artifact
Blender	Scene recipe, engine version, camera transforms, renders, masks/depth where configured	Blender runtime/image, asset licenses	Headless render and labeled sample E2E on designated node
Isaac Sim / Replicator	USD scene, synchronized RGB/depth/segmentation, sensor calibration, poses	RTX GPU with RT cores, Linux/runtime matrix	Actual rendered episode and metadata on compatible RTX worker
Isaac Lab	Parallel rollouts, observations, actions, rewards, done flags	Simulator version, GPU, config compatibility	Complete trajectory shard with episode boundaries
User uploads	Hash/metadata, parse result, safe preview	Virus/format screening; ZIP bomb safeguards	Test files uploaded to object store, parsed and versioned

Engine	Output contract	Constraint	Proof of real integration
Annotation model	Label, confidence, model hash, review provenance	Model capabilities, privacy	Prediction + editable reviewer correction persisted
HPC/Slurm	Job ID, state, logs, resource accounting, outputs	Slurm version, outbound agent, workload-compatible nodes	Submit, query, cancel, retrieve artifacts on configured cluster
Kubernetes GPU	Pod/job IDs, logs, metrics, output artifact	Node labels/drivers, quotas, image signatures	End-to-end workload on configured cluster

Hardware placement rule: An H100/H200 can be appropriate for inference, curation or model training but **cannot satisfy Isaac Sim's RTX real-time ray-tracing requirement** because A100/H100-class hardware lacks suitable RT cores. An engine-aware scheduler must reject unsuitable assignments with an actionable compatibility explanation. Validate the current Isaac Sim/Isaac Lab release matrix before pinning containers.

36.1 Additional future modalities

Structured tabular, event/time-series, spatial point clouds, 3D CAD/mesh, molecular/biological, simulation scientific fields, geospatial, multimodal documents, motion capture, and sensor fusion are first-class *schema extension points*. Do not claim scientific domain correctness without specialized validators, expert review and data policies.

37. Security and governance as product behavior

37.1 Authorizations

Authentication and tenant ownership must be enforced server-side for every dataset, run, key, webhook, export, sample and compute resource. RBAC baseline: Owner, Admin, Engineer, Labeler, Viewer, Service Account; scopes define exact operations (`datasets:read`, `datasets:write`, `runs:execute`, `compute:admin`, etc.). Resource-based policies may further restrict PII or licensed datasets. UI visibility is never a substitute for API authorization.

37.2 Dangerous boundaries

- Uploaded scripts and plugins: sandbox per job, outbound egress restrictions, no privileged host mounts, signed/pinned images, quota and time limits.
- Imports: SSRF and DNS-rebinding defense, deny metadata IPs/private ranges, content length and decompression bounds, MIME/sniff checks, malware screening policy.
- Provider credentials: stored in secret manager, encrypted, rotation supported; logs and UI must redact them; no secret values returned after creation.
- Compute agents: outbound connections, per-agent identity, revocation, lease expiry, signed job definitions, narrowly scoped temporary object-store credentials.
- AI-generated data: capture consent/rights/license and provider usage policy, prohibit unlicensed teacher-output training; avoid storing private teacher reasoning traces or sensitive contents unnecessarily.
- PII: classify/redact/approve as needed, data residency and deletion propagation, retention policies, immutable audit with minimized sensitive payloads.
- Abuse prevention: organization rate limits, model provider quota, spend caps, bot protection on key creation, adversarial prompt/poisoned data checks.

37.3 Governance UI

Present actionable source-use policies and approval requests before job dispatch. Show why blocked, who can approve, what would change, and actual provider/hardware restrictions. Every decision is auditable. Data card for each version includes purpose, sources, rights, quality measurements, known limitations, content risks, target training uses, and restrictions. Include human-signed review when required.

38. Quality, feedback, and evaluation

38.1 Quality profiles

Text: schema, grammar where applicable, executable verification, duplicated content, PII/rights and contamination checks, factuality estimates not guarantees. Vision: corrupt images, resolution, perceptual duplicates, object count, bounding box/mask validity, caption alignment, class distributions. Audio/video: codec integrity, silence/clipping, transcription alignment, temporal coherence, consent. Robotics: observation/action synchronization, sensor calibration, collision traces, reward semantics, units and episode completeness.

38.2 Quality gates

Every job can set minimum sample acceptance, maximum duplication, license coverage and validation thresholds. A failed gate publishes *nothing*, but stores an error report and allows an authorized correction/retry. Human review has inter-annotator disagreement and adjudication histories. Quality reports show denominators and coverage; confidence is not ground-truth accuracy.

38.3 Model impact

Accept evaluation feedback produced by external training systems. Keep holdout datasets access-controlled and prohibit generation workers from reading heldout solutions; measure changes on fixed real-world benchmarks; record baselines, confidence and cost; do not auto-approve synthetic-data iterations solely because they improve on self-generated tests. Use closed-loop generation with budgets, stop conditions and manual approval.

39. Pricing, product telemetry, and cost predictability

Meter with append-only usage events: dataset storage bytes and duration, data ingress/egress bytes, sample generation count, tokens/images/audio/video seconds, GPU-seconds per accelerator pool, simulation node-hours, annotation review units, API requests (if billable), custom provider passthrough charges, and export bytes. Distinguish actual external cost from estimated/internal charge. Show reserved, spent, pending and refunded amounts separately. Enforce hard caps before scheduling; later add warning thresholds and monthly organizational budgets. Provide pricing estimator with assumptions instead of pretending estimates are exact.

Admin telemetry: activation funnel (first dataset -> first run -> first published version -> first export), success rate per engine, time-to-dataset, quality acceptance rate, cost/sample, time in queue, run recovery success, frequent failure classes and churn indicators. Respect privacy and avoid recording raw prompts or dataset content into product analytics by default.

40. Feature parity and evidence policy

Release test for EVERY feature:

1. **Behavior:** Does the control perform the advertised mutation or query against a real authenticated backend?
2. **Persistence:** Does it survive refresh, reconnect, and backend restart?
3. **Authorization:** Does a different tenant/insufficient scope get denied?
4. **Failures:** Does a dependency failure display accurate status, retryability, and request ID?

5. **Accessibility:** Keyboard focus, labels, responsive layouts and reduced motion pass checks.
6. **API parity:** Can equivalent supported behavior be performed through a documented API/SDK call?
7. **Observability:** Can engineers trace the operation by ID and observe measured cost?
8. **Proof:** Is there a passing test, response/log reference, screenshot/video and known-limitations entry?

Each feature gets one of: NOT_STARTED, IMPLEMENTING, INTEGRATED, TESTED, RELEASED, BLOCKED_EXTERNAL. An engineer may NOT use RELEASED without evidence; BLOCKED_EXTERNAL cannot be marketed as live. DESIGN_ONLY is a separate truthful tag for screens/prototypes. CI should fail if unimplemented feature flags are enabled in production.

41. Codex operating model and governance

Codex is an implementation agent, not a substitute for real verification. The handoff ZIP contains AGENTS.md, CODEX_START.md, phased codex/phases/*.md, an acceptance matrix, and example contracts. Apply the source plan to an actual repository. Codex must inspect the repo before touching it, document baseline, implement the smallest runnable slice, run commands and tests, and record verified evidence in BUILD_LOG.md. Avoid one giant task to build everything in a single response. Finish phase gates before moving to optional systems; parallelize independent components only after stable contracts are accepted.

41.1 Mandatory review checkpoints

- Design tokens and first 5 screens: responsive screenshots, contrast, keyboard, empty/loading/error states.
- Core vertical slice: recorded end-to-end dataset flow using real Postgres/object store/workers.
- Security: negative cross-tenant tests, tokens, secret redaction, provider-license checks, budget enforcement.
- Generation/provider integration: authorized account and a redacted sample trace.
- Simulation: proof on compatible NVIDIA RTX or explicit BLOCKED_EXTERNAL.
- Release: passing CI, rollback, metadata restore exercise, runbooks and approved environment changes.

41.2 Explicit approvals

Always obtain user permission before: spending money, provisioning paid cloud/HPC, sending data to an external model provider, adding production secrets, changing live databases, publishing a public deployment, registering paid domains, or altering an existing production system. Local synthetic test data and no-cost tooling may be used as configured, but never claim access to unconnected resources.

41.3 Required Codex output per phase

Summary, Files changed, Commands executed, Passing tests, Screenshots or traces, API examples, Verified features, Known blockers, Security assessment, Next phase and Approval requested (if any). Include failures and incomplete steps; never replace genuine errors with fabricated sample results.

42. Build milestones with executable acceptance gates

Phase	Implementation	Demo gate (must pass before phase marked done)
P00	Baseline, architecture decisions, repo, CI, env examples	Fresh clone setup, app shell/API boot, health checks and tests

Phase	Implementation	Demo gate (must pass before phase marked done)
P01	Geist-inspired tokenized UI shell + responsive routing	Desktop/mobile screenshots; working navigation, auth placeholders correctly disabled
P02	Multi-tenancy, auth, PostgreSQL models, object store	Real sign-in; no cross-tenant reads; migrations/seed idempotent
P03	Dataset + version + multipart upload APIs and screens	Upload fixture; hash verified; browse after refresh; preview/download
P04	Recipe schema, run queue, orchestration and CPU worker	Submit -> process -> validate -> immutable publish -> export
P05	OpenAPI, SDKs, CLI, signed webhooks, audit	API/SDK/CLI same end-to-end job, signed webhook verified
P06	Quality/curation and annotation with reviews	Dedupe/label/edit/approve; failed gate refuses publish
P07	Authorized teacher provider + multi-modal adapters	Real licensed provider sample; permissions/budget/usage and retry validated
P08	Compute pools/agents, scheduler, RTX/Slurm adapter	Node capability checks; real eligible job or honest blocked status
P09	Blender, Isaac Sim, Isaac Lab runtime integration	Real designated worker produces inspectable episode/sample artifacts
P10	Evaluation, feedback, search and dataset diffs	Fixed benchmark comparison and reproducible version diff
P11	Enterprise, billing, tenancy hardening, ops	SSO/test policies, audits, billing reconciliation, restore drill
P12	Load, disaster recovery, security review, release	Full regression and approved staging cutover; no critical defects

Phases represent dependency-ordered scope, not credible calendar promises. First P00-P05 is the first genuinely working platform. Remaining phases extend real production capability; all are planned unless verified as implemented in a live repository.

43. First five Codex sessions: precise work orders

Session 1: Repository baseline and foundation. Read AGENTS.md, plan and docs. Set up pnpm/Next.js/FastAPI/PostgreSQL/MinIO/worker/Docker Compose, .env.example, Makefile and CI. Create

health endpoints and first DB migration, seed fixture and two basic smoke tests. Show actual boot logs, test result and file tree. Do not claim other modules implemented.

Session 2: Design system and app shell. Build original Vercel-inspired UI tokens, typography, nav, project switching, command palette, theme mode, responsive layouts, data table, form state and skeleton/empty/error patterns. Connect dashboard to health and tenant-aware API; hide absent modules. Validate at 360/390/768/1440 and keyboard/axe. Provide screenshots.

Session 3: Dataset CRUD and storage. Organization/project models, scoped user auth, create/list/get dataset, presigned multipart upload, completion checksum, sample/object metadata, search/filter/pagination, dataset detail. Tests: file upload, refresh persists, revoked access, cross-tenant denial, invalid content, duplicate request.

Session 4: Full job vertical slice. Compile JSON recipe, create idempotent run, durable queue, CPU generator/validator, artifact manifest with hashes, quality gate, immutable publish, export, structured logs and retry. Demonstrate a real 100-sample job; test worker crash, duplicate idempotency, budget denial and invalid data.

Session 5: Developer parity. Generate OpenAPI from API, typed client and Python/TypeScript SDK, CLI, webhook signing and delivery/replay, accurate docs, integration and Playwright E2E. Run same workflow by SDK and UI and verify the manifest hashes/cost ledger. End with a production readiness gap report.

44. Product-wide acceptance scenarios

A: Text dataset end-to-end. Sign in -> create text dataset -> configure a deterministic approved local CPU generator -> launch -> witness trace events -> quality report -> publish immutable version -> download JSONL/Parquet -> compare hashes -> API/SDK repeat -> webhook proof. Expected: persisted results and no fake UI.

B: Image upload + labeling. Upload signed real image bytes -> thumbnails generated -> annotation task created -> auto label or manual label -> reviewer edits -> export COCO/YOLO -> audit shows edits and user -> quality gate blocks corrupt annotations. Expected: preview and exported labels agree.

C: Teacher synthesis. Connect a *permitted* provider -> record rights and quota -> prompt+schema validation -> generate -> verify against heldout/task-specific tests -> reject invalid samples -> store response lineage -> export. If no authorization/credentials, show clearly blocked test, not a fake success.

D: Blender / Isaac. Compatible remote worker detected -> validate scene and assets -> budget approval -> job queued -> execute -> synchronized frame/label bundle -> inspect episode -> export. If an RTX node is missing for Isaac Sim, server returns a structured `CAPABILITY_UNAVAILABLE` with requirements; UI shows blocking message and never launches.

E: Multi-tenant attack. Two organizations: neither can read or mutate the other's datasets, logs, keys, upload URLs, artifacts, webhooks, compute jobs or billing. Include ID substitution and cursor/pagination tests. 403/404 policy consistent and no sensitive leakage.

F: Failure, recovery, and billing. Kill worker, duplicate requests, provider 429, storage offline, webhook 500, quality failure, budget cap. System responds accurately; lease expires; retries are idempotent; published versions are not corrupt; cost ledger reconciles and no double-billing occurs.

G: Responsive developer UI. 360px mobile to wide desktop, light/dark, keyboard-only actions, screen reader checks, real data/empty/slow/error cases, keyboard command palette, API docs; no dead controls, overflow or hidden critical action.

45. Detailed delivery checklist and stop conditions

A phase is blocked if: migrations do not apply cleanly; test fixture cannot run from a new checkout; any advertised page has dead controls; authentication is bypassed; credentials appear in client assets or logs; tenant-isolation tests fail; data can be published without a passing quality/license gate; user cannot cancel a paid run; a GPU integration is

simulated as real; API docs do not match running API; monitoring cannot diagnose a failed run; or CI has failing critical tests.

A general availability candidate also needs: threat model review; penetration test remediation; disaster recovery drill; backup restore evidence; documented key/secret rotation; alert escalation; on-call runbooks; audit retention and deletion; explicit supported-engine version matrix; license/legal check; accessibility testing; production load tests; cost model and spend caps; product terms/privacy; signed artifact provenance; rate/abuse limits; incident communications; safe feature flag rollback; and zero known critical vulnerabilities.

46. Decision summary for Codex

1. **Start with visual architecture:** Original Geist-inspired theme, UI kit and app shell before product screens, but connect to real backend services immediately.
2. **Vercel is for the web product:** Keep the web on Vercel; run FastAPI, durable workflows, storage, and CPU/GPU/HPC workers on appropriate container/cluster infrastructure.
3. **Every interface uses one API contract:** Web, CLI, SDK, automation and AI agent call the same policy-enforced backend.
4. **Make the core dataset flow work before specializing:** Create/import -> recipe -> run -> verify -> version -> export -> webhook.
5. **Add multimodal engines as audited adapters:** Provider permissions, asset rights, capability checks, and independent verification are always explicit.
6. **Show the truth:** Production features show real data, mock/demo fixtures are clearly marked, unavailable modules are disabled or marked blocked.
7. **Do not spend or deploy without approval:** PR/preview development is preferred; external paid calls and production promotions require explicit review.
8. **Record proof:** A Codex phase is complete only after run IDs, screenshots, test logs, contracts, and known limitations are saved.

Updated reference links (verify versions before implementation)

- Vercel Geist: https://vercel.com/geist/introduction ; typography: https://vercel.com/geist/typography
- Vercel Functions: https://vercel.com/docs/functions/configuring-functions/duration ; feature availability and limits may differ by plan and date.
- Codex AGENTS.md guidance: https://learn.chatgpt.com/docs/agent-configuration/agents-md
- Codex iterative engineering harness: https://developers.openai.com/cookbook/examples/codex/iterating-development-workflows-with-codex
- NVIDIA Isaac Sim technical requirements: https://docs.isaacsim.omniverse.nvidia.com/6.0.0/installation/requirements.html
- OpenAPI: https://spec.openapis.org/oas/latest.html ; CloudEvents: https://cloudevents.io/
- OWASP API Security Top 10: https://owasp.org/API-Security/ ; WCAG 2.2: https://www.w3.org/TR/WCAG22/